

TD9 B3 : Protéger une application web en appliquant un codage sécurisé.

Table des matières

1) Problématique.....	1
2) Etapes	1
3) Conclusion.....	5

1) Problématique

Nous souhaitons suivre une formation sur le thème de la sécurisation des applications Web, durant cette formation nous allons jouer le rôle du hacker et celui du développeur. Nous nous aiderons des vidéos pour réaliser les étapes.

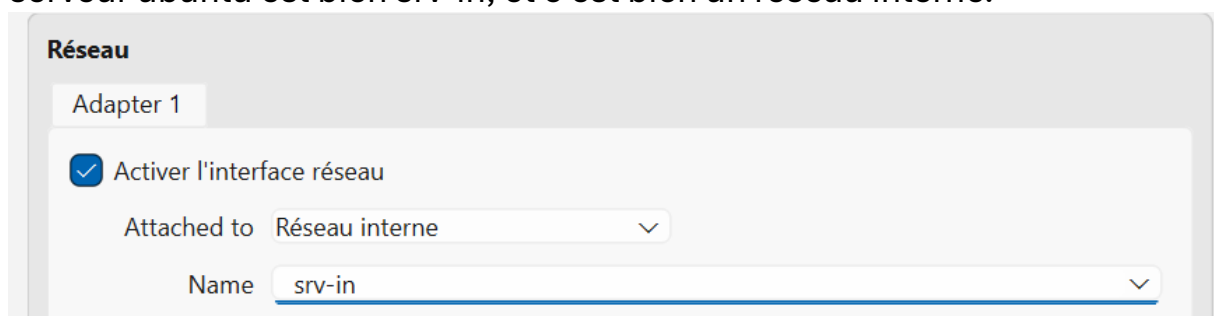
2) Etapes

Etape 1 :

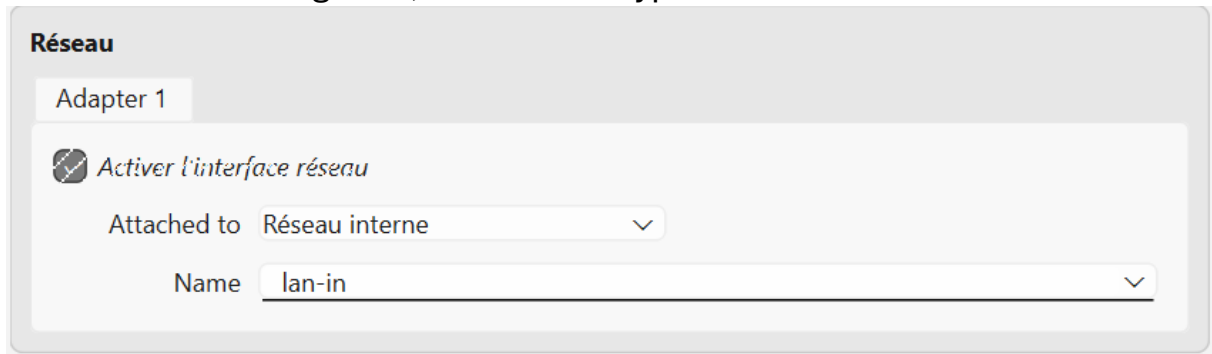
1. Il faut avoir VirtualBox sur son PC, avoir minimum 6 GO de mémoire et installer les 4 machines virtuelles du lien suivant : lienmini.fr/6988-601. Lorsque les VMs sont installés on doit les ouvrir avec VirtualBox pour les ajouter automatiquement.

2. Nous allons suivre les étapes de la vidéo LAB-THEME4-CONFIG présente sur le bureau de la vm ubuntu légitime.

On nous demande de vérifier d'abord les noms des cartes réseau, celle de serveur ubuntu est bien srv-in, et c'est bien un réseau interne.



Pour la VM client légitime, le nom et le type de réseau est bon aussi :



Réseau

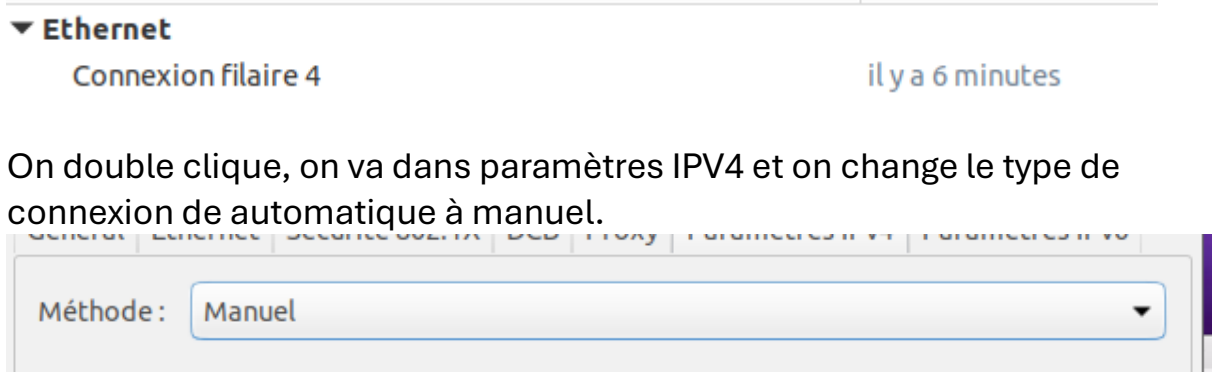
Adapter 1

☒ Activer l'interface réseau

Attached to: Réseau interne

Name: lan-in

Dans la VM client légitime, nous devons modifier les paramètres IPs. On nous demande de modifier la connexion la plus récente, la mienne est la 4 :



▼ Ethernet

Connexion filaire 4 il y a 6 minutes

Méthode: Manuel

On ajoute l'adresse suivante puis on enregistre :

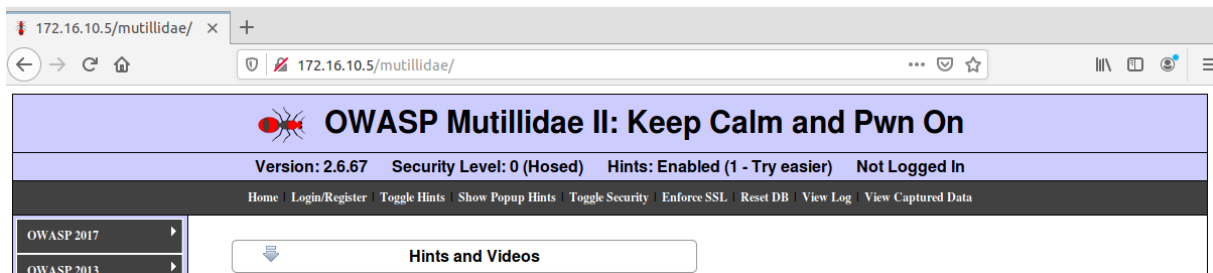
Adresse	Masque de réseau	Passerelle
192.168.50.10	24	192.168.50.254

Après cela on désactive puis réactive le réseau.

On ouvre LXTerminal, on vérifie la connexion avec la commande **ip a s**

```
prof@prof:~$ ip a s
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d8:63:73 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.10/24 brd 192.168.50.255 scope global noprefixroute enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::af3a:837c:9a40:62fa/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Puis on ouvre Firefox et on teste la connexion :



Ça fonctionne on accède bien au serveur.

On vérifie que le pare feu est bien en accès par pont :

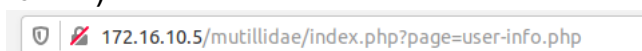


Ensuite, on va dans le mode expert puis on vérifie que adapter 2 a pour nom lan-in et le 3 srv-in.

Etape 2 :

3. Pour cette étape, j'ai dû configurer le réseau de la VM du client HACKER pour qu'il puisse se connecter au localhost.

Ensuite on accède à la page user-info via ce lien (ou via les instructions sur la VM) :



Dans la page de connexion il faut utiliser cet identifiant :

Sur la page d'authentification, saisir les informations suivantes :
login : harry
mot de passe : 'or 'a' = 'a

On tombe sur le résultat suivant :

[Don't have an account? Please register here](#)

Results for "harry".23 records found.

Username=admin
Password=adminpass
Signature=g0t r00t?

Username=adrian
Password=somepassword
Signature=Zombie Films Rock!

Username=john
Password=monkey
Signature=I like the smell of confunk

Username=jeremy
Password=password
Signature=d1373 1337 speak

Username=bryce
Password=password
Signature=I Love SANS

Donc on constate que lorsque le site est de niveau 0 (aucune protection), l'injection SQL nous permet d'accéder aux informations confidentielles du site mutillidae et donc cela montre que le site n'est effectivement pas protégé contre les injections SQL.

Etape 3 :

4. Désormais nous allons augmenter le niveau de sécurité du site à 5 en cliquant sur Toggle Security deux fois :

Security Level: 5 (Server-side Security)

[Login/Register](#) | [Show Popup Hints](#) | [Toggle Security](#) | [En](#)

On réessaye l'injection SQL et cette fois-ci une alerte s'affiche :

Dangerous characters detected. We can't allow these. This all powerful blacklist will stop such attempts.

Much like padlocks, filtering cannot be defeated.

Blacklisting is l33t like l33tspeak.

OK

On peut donc constater que les mesures prises par le site pour éviter les injections SQL lorsque le niveau de celui-ci est de 5 sont bien

fonctionnelles grâce au système de liste noire et grâce à la vérification de la longueur des données saisies.

5. En comparant le code source des deux sites, on s'aperçoit que l'une des fonctions est probablement la fonction qui permet d'empêcher les injections SQL, dans la version sécurisée du site, la fonction est en TRUE

```
<script type="text/javascript">  
  var lValidateInput = "TRUE"
```

Et dans la version sécurisée non :

```
<script type="text/javascript">  
  var lValidateInput = "FALSE"
```

La version sécurisée active le script de filtrage des caractères spéciaux avant l'envoi du formulaire.

On peut donc déduire qu'en matière de bonnes pratiques, il est absolument nécessaire d'ajouter un script de filtrage des caractères spéciaux dans son code pour éviter les injections SQL sur son site.

3) Conclusion

En conclusion, grâce à la formation de Cibecco j'ai compris comment marchait les injections SQL, et comment s'en défendre.